

C 与 Java 点乘性能横向对比实验报告

Course: CS219: Advanced Programming

Project: Project 2

Author: 李语尚

Date: 2026 年 4 月 12 日

目录

1 前言	3
2 实验设置	3
2.1 测试任务	3
2.2 对比维度	3
2.3 数据一致性	3
3 结果与分析	4
3.1 对比 1: C 无优化 vs JDK 17 无优化	4
3.2 对比 2: C 无优化 vs C -O3	4
3.3 对比 3: JDK 17 vs JDK 26	5
3.4 对比 4: JDK 26 无优化 vs 优化方案	5
3.5 对比 5: C -O3 vs JDK 26	6
4 结论	6
5 参考文献	6

1 前言

本项目聚焦于向量点乘（dot product）场景下的跨语言性能对比，核心问题是：在相同任务和相同规模输入下，C 与 Java 的执行效率差异来自哪里。

为了让对比结果更有解释力，本报告不仅给出图表，也结合编译优化、JIT 机制、并行开销和内存带宽约束进行分析。

2 实验设置

2.1 测试任务

统一测试任务为数组点乘：

$$\text{sum} = \sum_{i=0}^{N-1} a_i b_i$$

该任务理论复杂度为 $O(N)$ ，因此不同实现的性能差异主要来自常数因子、编译优化和运行时策略。

2.2 对比维度

本报告包含 5 组核心对比：

- C 无优化 vs JDK 17 无优化
- C 无优化 vs C -O3
- JDK 17 vs JDK 26（无额外优化）
- JDK 26 无优化 vs JDK 26 优化方案
- C -O3 vs JDK 26

2.3 数据一致性

为避免“数据生成”阶段成为瓶颈或引入偏差，C 端使用轻量且统计特性良好的 PCG32 伪随机数算法：

```
1 // 1. 状态推进：64-bit LCG
2 rng.state = oldstate * 6364136223846793005ULL + (rng.inc | 1);
3
4 // 2. 输出置换：XSH-RR
5 uint32_t xorshifted = ((oldstate >> 18u) ^ oldstate) >> 27u;
6 uint32_t rot = oldstate >> 59u;
7 return (xorshifted >> rot) | (xorshifted << ((-rot) & 31));
```

3 结果与分析

3.1 对比 1: C 无优化 vs JDK 17 无优化

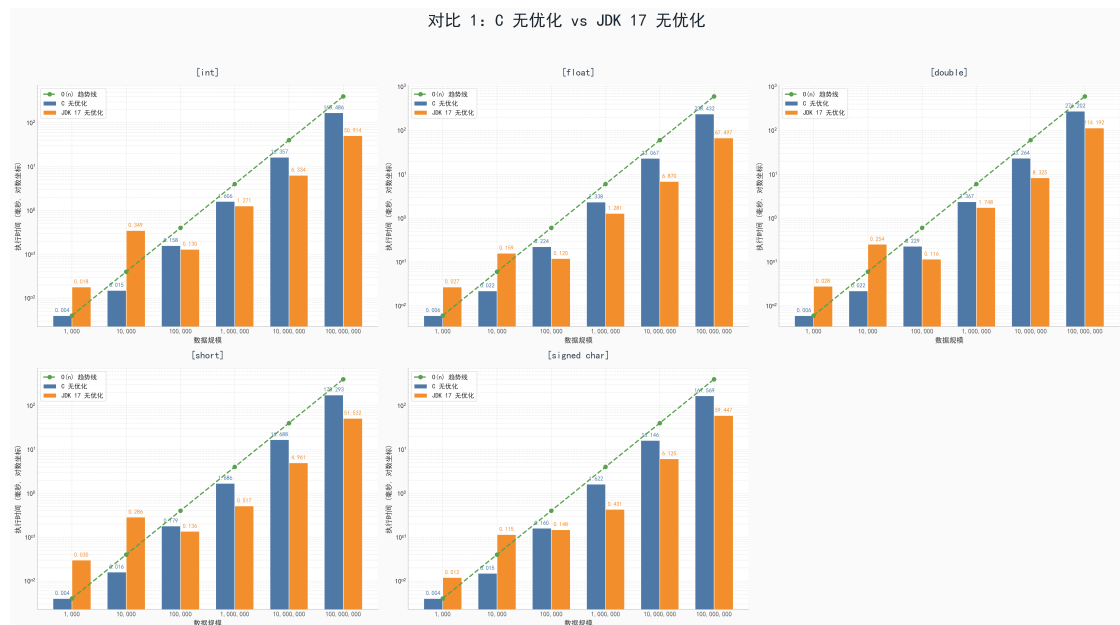


图 1: C 无优化 vs JDK 17 无优化

两者都呈线性增长趋势。在大规模输入时，JDK 17 由于运行时热点优化，性能并不一定弱于 C 的无优化版本。

3.2 对比 2: C 无优化 vs C -O3

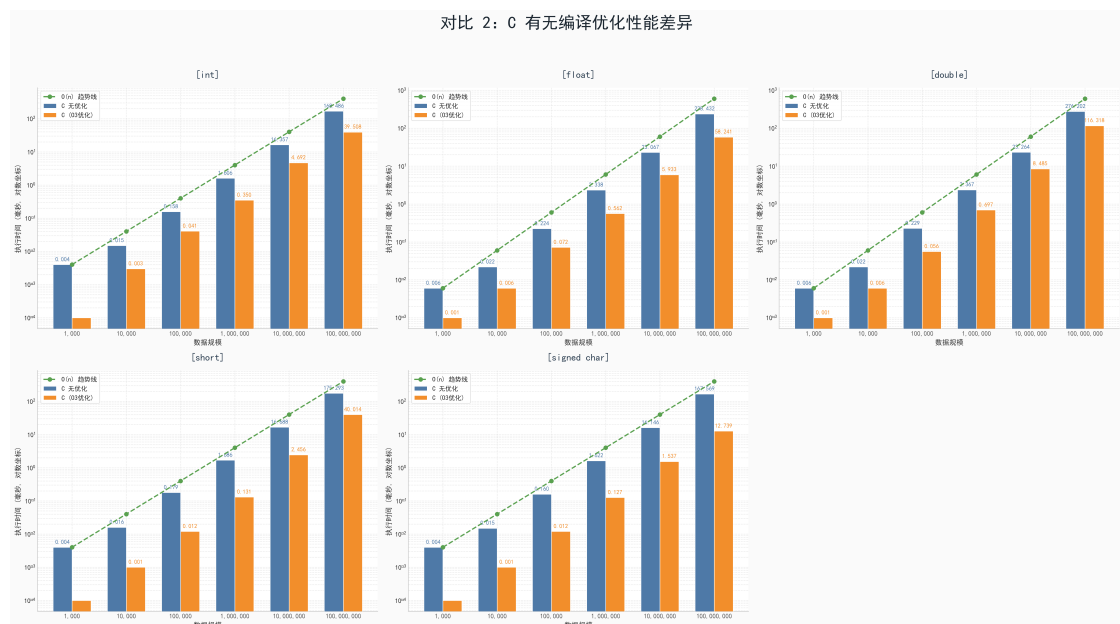


图 2: C 无优化 vs C -O3

-O3 带来的加速非常显著，主要来源于自动向量化、循环展开和寄存器分配优化。该组结果说明编译器优化对数值密集循环影响极大。

3.3 对比 3: JDK 17 vs JDK 26

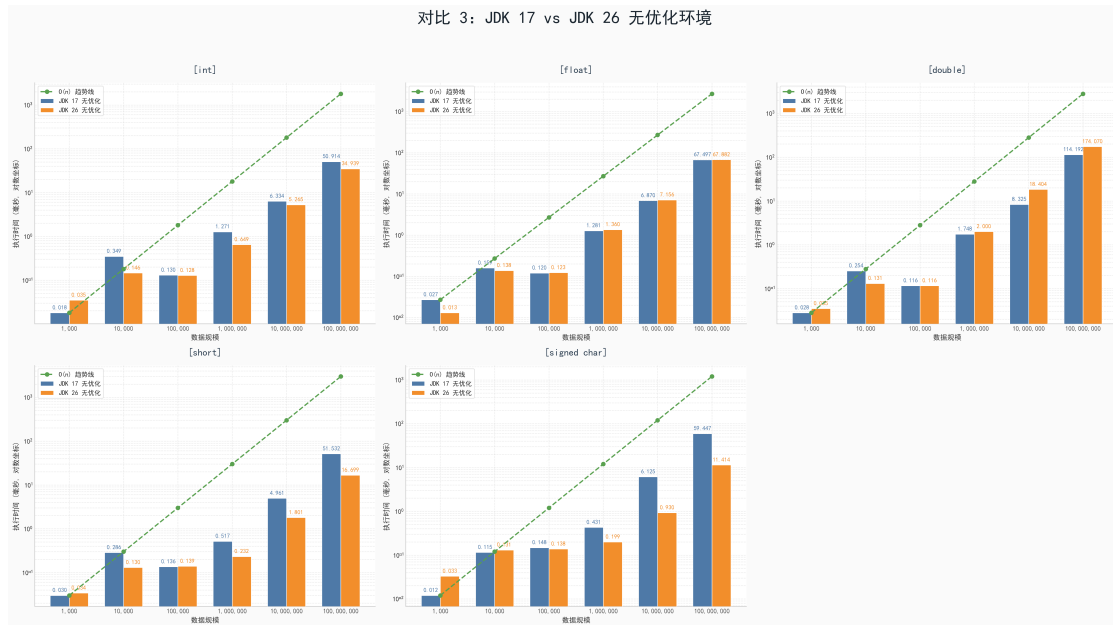


图 3: JDK 17 vs JDK 26 (无额外优化)

JDK 26 在多数规模下优于 JDK 17，体现了新版 JIT 在代码生成和热点优化上的持续演进。

3.4 对比 4: JDK 26 无优化 vs 优化方案

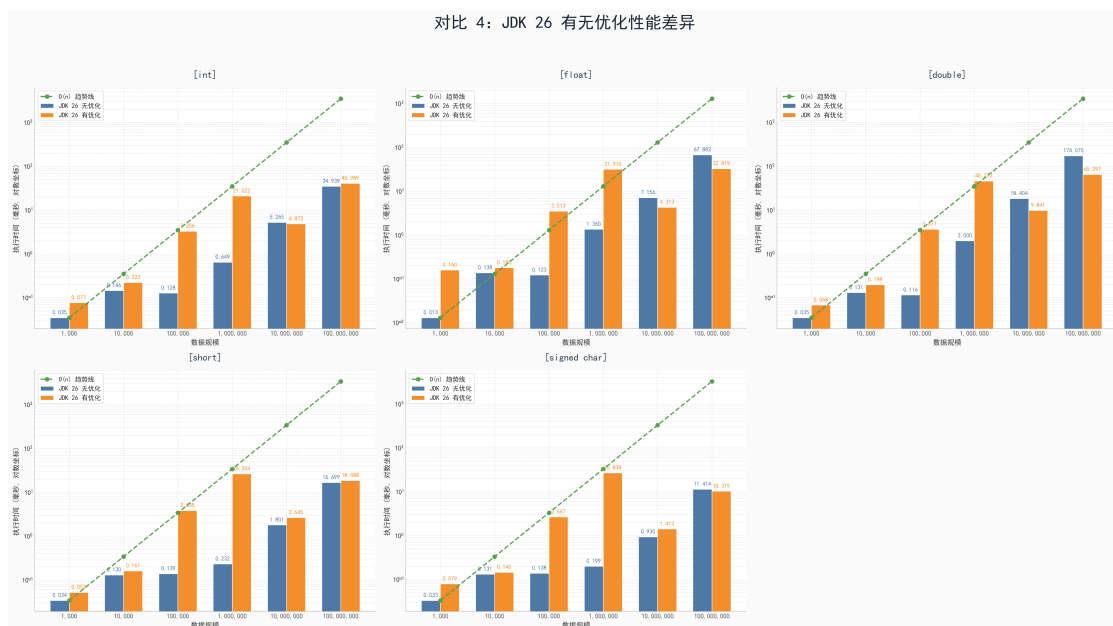


图 4: JDK 26 无优化 vs 优化方案

并行方案并非总能提速。点乘是内存带宽敏感任务，线程调度、对象封装与缓存争用可能抵消并行收益，甚至出现回退。

3.5 对比 5: C -O3 vs JDK 26

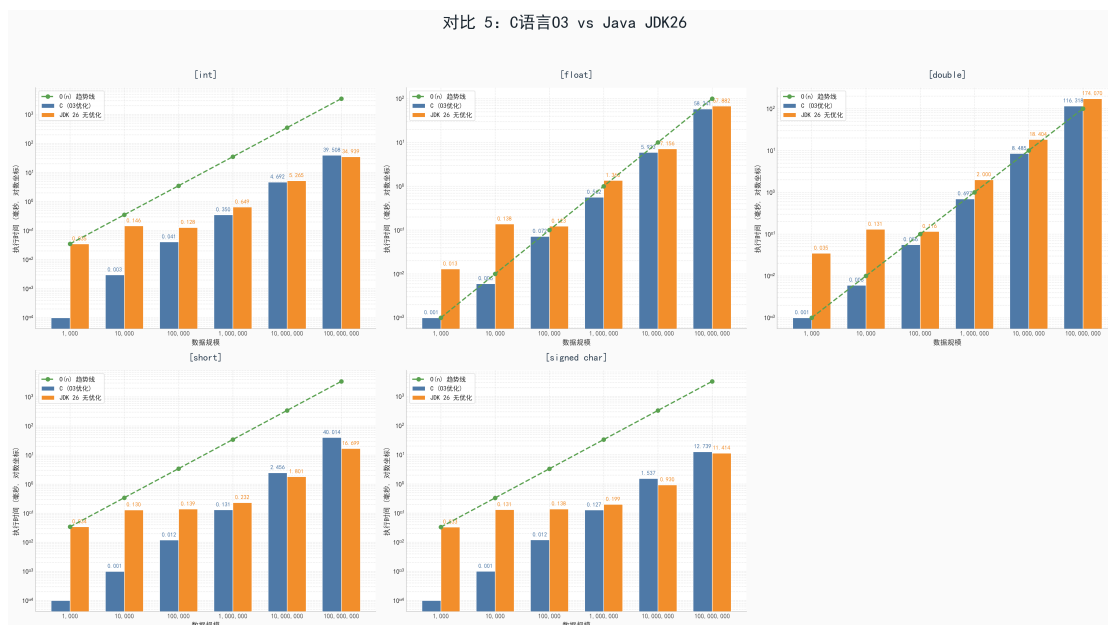


图 5: C -O3 vs JDK 26

C 的 AOT 静态优化与 Java 的 JIT 动态优化分别走了不同路线：前者稳定、后者灵活。在部分规模下，Java 可接近甚至追平高优化 C 版本。

4 结论

- 点乘任务整体符合线性增长，差异主要来自常数项。
- C 开启 -O3 后性能提升巨大，应作为 C 侧基线配置。
- JDK 版本升级可带来可观红利，JDK 26 的表现明显更强。
- 并行优化需基于基准测试决策，不可默认“多线程必然更快”。

5 参考文献

参考文献

- [1] Knuth, D. E. (1998). *The Art of Computer Programming, Volume 2: Seminumerical Algorithms* (3rd ed.). Addison-Wesley.
- [2] O'Neill, M. E. (2014). PCG: A Family of Simple Fast Space-Efficient Statistically Good Algorithms for Random Number Generation.
- [3] OpenJDK Documentation. Retrieved from <https://openjdk.org/>